A Longitudinal Clinical Decision Engine for Prescription Safety

Prachi Lund, Bhoomi Chhabria, Mahek Hingorani, Navya Rangwani
Department of Computer Engineering, Vivekanand Education Society's Institute of Technology
Mumbai, India

Abstract- In Healthcare systems where patient's health records are distributed among several providers, prescription errors and delayed disease progression detection are major issues especially in resource constrained environments like India. This paper presents MERIDIAN, a long-term decision engine that merges patient medical history into a structured time-ordered clinical timeline and utilizes rule-based analytical modules to verify prescription safety. MERIDIAN utilizes demographics, diagnoses, medications and laboratory reports including scanned or image-based reports processed through the Tesseract OCR engine and arranged in chronological order to perform trend-based clinical reasoning. A Python-based decision engine scans for drug-drug interactions screens for drug allergies analyzes lab-drug contraindications detects duplicate tests and flags overdue tests combining the results into a red-flag risk summary for clinicians. MERIDIAN is built with the stack (React.js, Node.js, Express.js, MongoDB) for the application layer utilizes Firebase Authentication for role-based access control operates a Flask/FastAPI microservice for analytical processing and Nodemailer to send automated notifications. This paper describes the architecture, core modules and underlying technologies of MERIDIAN. Outlines the evaluation plan to test its accuracy and clinical usability with synthetic chronic-disease datasets. MERIDIAN aims to cut diagnostic testing lower harmful drug events and supports smoothness of care in multi-provider healthcare ecosystems.

Keywords—Decision Support System, Longitudinal Health Records, Drug Interaction Detection, Optical Character Recognition, Prescription Safety, MERN Stack.

I. INTRODUCTION

As we currently know, electronic health record (EHR) systems in the clinical domain are passive stores of information without providing decision making aid. Physicians mostly refer only to the report when they prescribe medications, since the earlier diagnoses, drugs and lab results are typically scattered in paper records and fragmented care providers. This fragmentation not only increases the chances of drug interactions slows down the detection of progressing diseases but also introduces unnecessary tests that raise the cost and risk to the patient.

Such a scenario is more severe for chronic diseases such as diabetes and hypertension that require continual tracking of lab trends and long-term medication plans. At a single point in time a report can neither show the progression of kidney function between patient visits, nor detect new drug interactions with an existing regime nor determine if a test has already been done elsewhere. There is a need for something that considers history as a continuum of analyzable timelines rather than a set of isolated documents.

The current healthcare record systems in paper-based and multi-provider settings present several common shortcomings: (i) patient medical data is fragmented and not in a timeline; (ii) past medicines and allergy history are not available when prescribing; (iii) lab trends over time are not considered; (iv) drug–drug and drug–disease interactions are at risk; (v) diagnostic tests are often repeated raising the patient cost; and (vi) there is no systematic way to remind doctors about overdue monitoring tests for chronic-disease management.

To address these shortcomings, this paper introduces MERIDIAN, a clinical decision engine that transforms scattered medical records into structured time-ordered clinical timeline with a rule-based analytical engine on top that aids doctors in prescribing safely. The rest of the paper is organized as follows. Section II reviews the related work on the domain of decision support and electronic health records. The functional modules and the technology stack used to build MERIDIAN are discussed in Section III. The system architecture and the end-to-end data flow are presented in Section IV. The evaluation methodology that is expected is discussed in Section V. Finally, Section VI concludes the paper, highlights points to future work.

II. RELATED WORK

Clinical decision support systems (CDSS) have been the subject of extensive research to improve healthcare decision-making processes for safety and efficiency. As defined by the Agency for Healthcare Research and Quality, CDSSs are "decision support tools that provide clinicians with information and knowledge to aid decision-making during patient care" [1]. Sutton et al. have investigated the duality of the advantages and disadvantages of clinician decision support [2]. The strengths included the ability of clinicians to rely on medical information and the seamless incorporation into the existing practice. Meanwhile, issues such as an overload of alerts and the complexity of the software present challenges.

The clinical information infrastructure is one of the primary components in the construction of decision support systems because it allows for the digital compilation of medical records for storage and retrieval. Previously published literature and training resources on EHRs and health information exchange discuss the role of infrastructure in supporting the seamless health information exchange between facilities to maintain continuity of care [4]. However, despite the presence of electronic health records, the chronological consistency of patient records is not guaranteed, especially in

hospital settings where paper-based prescriptions, laboratory test results, and other documents are involved.

Another interesting prospect in clinical information infrastructure is the conversion of documents into machine-encoded data for analysis. An example of such technology is the application of OCR tools, such as Tesseract, to extract text from document images for further processing [7]. In the context of clinical laboratories, this technology can be used to mine relevant information from laboratory test result documents. However, additional steps are required to process and validate the information before it can be incorporated into a structured format suitable for compilation with other documents, such as prescriptions. This includes the additional step of analysis for the accuracy of the extracted data.

A major aspect of clinical information infrastructure involves the enhancement of medication safety. Clinical information technology tools have been studied in the context of medication preparation processes with promising results in terms of reducing medication errors [5]. Conventional tools for medication administration rely on cross-referencing medication orders with specialized databases. MERIDIAN builds upon these concepts by considering medication histories, allergies, laboratory test results, medication-related laboratory tests, and follow-up instructions.

When considered together, these aspects of clinical information infrastructure present with an opportunity to develop a comprehensive decision support tool that utilizes each component. As seen in the described EHRs and CDSSs, there is a current practice of utilizing existing software to manage and store medical information while supporting clinicians during point-of-care decision-making. The MERIDIAN system is notable for its comprehensive approach to clinical decision support. From the perspective of a clinician seeking to prescribe medication or order diagnostic imaging, the system offers the synthesis of disparate pieces of information, including medication histories, test results, and patient timelines encoded with relevant documents for review. Additionally, the medication safety features of the system help to reduce errors stemming from the omission of pertinent information, such as patient allergies. The described clinical information technology tools can be combined into a workflow that supports clinicians during medication ordering and test ordering by presenting the chronological context of medication orders and laboratory tests as well as detecting repeated or overdue tests.

TABLE I
COMPARISON OF TRADITIONAL EHR/CDSS APPROACHES WITH MERIDIAN

Capability	Traditional EHR / Generic CDSS	MERIDIAN
Data view	Single encounter / latest report	Chronological, longitudinal timeline
Source data	Assumes pre-digitized records	Digitizes scanned reports via OCR
Interaction check	Current prescription only	Cross-referenced with history and lab trends
Duplicate tests	Not addressed	Automatically detected
Preventive care	Not addressed	Overdue-test reminders
Output	Static record view	Aggregated red-flag risk summary

III. SYSTEM DESIGN AND IMPLEMENTATION

A. Technology Stack

MERIDIAN is organized as a web platform which operates in conjunction with a Python analytical microservice. I envision the end utilizing React.js for the creation of role-specific dashboards, including those pertinent to patients, doctors and receptionists, and incorporating modules for appointments, prescriptions, medical records and an AI-assisted chat. The backend is engineered using Node.js and Express.js and offers RESTful APIs for the purposes of login and role control, user, doctor and patient management, appointment scheduling, prescription, medical-record processing, OCR coordination, an AI chatbot API, notification issuing via Nodemailer and dashboard report generation. Firebase Authentication enables sign-in, sign-up and role assignment for the three user types. MongoDB Atlas stores data within collections for users, patients, doctors, appointments, medical records, prescriptions, chat history and system logs. Laboratory report digitisation and analysis are undertaken by a Python microservice based on Flask/FastAPI while Chart.js and Recharts visualize trend charts and dashboard reports, on the end.

B. Core Functional Modules

The analytical capability of MERIDIAN comprises a suite of modules that work in tandem as depicted in the block diagram of the system architecture. The OCR engine, which is powered by Tesseract, extracts text from prescriptions and lab reports uploaded as PDFs or images. A data-cleansing step follows in which units of measurements are standardized, test names replaced with codes, and sanity checks of values performed. Thereafter, the data are archived in the long-term database by the long-term data-builder module, which organizes the information about a patient's diagnoses, medications, and tests in chronological order, thus enabling the construction of a time series of each relevant variable. The module also raises flags for abnormal lab values.

Above the timeline, six analytical modules operate: a lab trend analyzer that scrutinizes successive lab readings for abnormal trends; a drug–drug interaction engine that cross-references new prescriptions against a predefined interaction database; a drug–allergy checker that compares prescribed medications with patient-recorded allergies; a lab–drug contraindication engine that identifies medications incompatible with abnormal lab values; a duplicate test detector that recognizes diagnostic tests performed in the past at other facilities; and an overdue test engine that reminds clinicians of tests due for monitoring.

The outputs of these modules are collated by a risk aggregator that computes a severity score, and a decision simulator that evaluates the safety of proposed prescriptions before finalization. A red-flag generator compiles all findings in an alert panel for the prescribing clinician, thus completing the loop, with a feedback channel for prescription overrides in response to a flagged risk.

C. Data Model

The MongoDB Atlas database is organized into 8 collections which together support the view required by the analytical modules. MongoDB Atlas database stores the user's profiles with their authentication data in the Users collection. The Patients and Doctors collection store the users' professional records with additional fields for a doctor's specialization and a patient's health indicators. MongoDB Atlas database also stores the schedules of the appointments between the patients and the doctors in the Appointments collection. The Medical Records collection stores medical-related notes with timestamps to mark the date of a record. Finally, the Prescriptions collection stores the list of drugs, the dose quantity, and the date of a prescription. The interaction and contraindication engines query against these entries. The MongoDB Atlas database has a Chat History collection which persists conversations with the AI assistant for continuity across sessions. The MongoDB Atlas database also has a Logs collection which records system and audit events. Storing each entity as a timestamped document rather than as a flat file allows the longitudinal data builder to efficiently query a patient's full history using indexed fields such, as patient identifier and record date.

D. Algorithmic Approach

The core analytical modules are all based upon a rule-based mechanism. Each module takes the relevant part of the patient's history and checks for matches against a reference set or set limits, issuing a structured alert that gets processed by the risk aggregator. For the drug–drug interaction engine, every pair of medicines in the patients list including the new medicine being added are checked against a list of drug interactions. Those matching an interaction pair on the list are issued as alerts with severity. The test detector compares every new test request against the patient's test results within a timeframe, such as 30 to 90 days depending on the test. Any matches found in this duration are flagged as duplicates prior to scheduling. The overdue test engine detects tests that are overdue by analyzing the time that has passed since the recommended test was performed. It issues a reminder if the time exceeds a certain threshold. The lab trend analyzer analyzes test results over time looking at percentage changes and trends and identifies any outliers or rapid changes. All the alerts are then collated by the risk aggregator which summarizes them into severity. The decision simulator re-applies all the checks to a new medicine before it is saved to the patient's history thus giving the doctor a warning before the medicine becomes a part of the patient's record.

IV. SYSTEM ARCHITECTURE

Fig. 1 Depicts MERIDIAN's architecture. The Users/Clients layer accommodates three roles: patients, doctors, and receptionists.

Patients can book, view doctors, view records, prescriptions and utilize the AI assistant. Doctors manage appointments, patients, records, prescriptions, OCR reports and the AI medical assistant. The Receptionists handle patient and doctor records, schedule appointments, generate reports. All the roles access the system via Web Apps, Desktops, Tablets or Mobile devices.

The Frontend layer is based on the React.js framework and provides a dashboard-like experience, which is personalized according to the user's role. It comprises appointment and prescription views, screens for details and medical records, an interface for uploading and reviewing OCR documents and a chatbot widget driven by AI.

Every frontend request passes through the Authentication layer, which is implemented with Firebase Authentication, handling logins, registrations and role management, before reaching the Backend layer, which is implemented in Node.js and Express.js.

The Backend comprises of authentication and role-based authorization middleware,users/doctors/patients' management, appointment and medical record management, prescriptions handling, OCR processing, AI chatbot API, and email notification service (implemented with Nodemailer). The Backend also exposes Dashboard, Reports, and File Endpoints.

The Database Layer is based on MongoDB Atlas, storing all data in separate collections for users, patients, doctors, appointments, medical records, prescriptions, chat history, and logs.

The Backend exposes API Endpoints to the External Services Layer. This Layer comprises an AI/LLM service (which provides knowledge, summarization, and QA capabilities), and Nodemailer (used for appointment-related notifications and emails).

An OCR/Analytical microservice (implemented in Python) receives document upload requests from the Frontend, performs image preprocessing (noise removal), OCR, and data parsing tasks before persisting the results in a database and exposing them to analytical tools (described in Section III).

The application's security and access control policies are implemented at multiple layers. Firebase Auth provides signed tokens, which are validated by the Express middleware on every API call. Role-based access control (RBAC) restricts authorized endpoints; for instance, only doctors and receptionists can view another patient's prescription history. The Frontend, Backend, and OCR/analytical microservice communicate with each other via internal authenticated API calls. Such an approach adds security since the OCR and analytical services are separate instances.

The separation of concerns and layer division contribute to scalability: Adding more OCR/analytical microservices behind

the load balancer would provide horizontal scaling without changing the Frontend or Database components. This approach is beneficial because the OCR/analytical microservice is stateless and relies on shared database resources (MongoDB Atlas). Therefore, increased traffic or additional patients would not require frontend adjustments or database scaling.

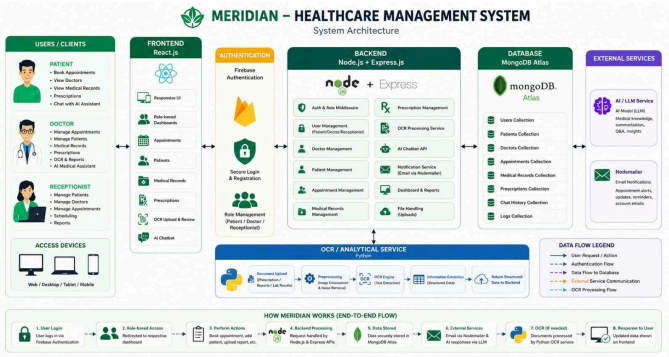


Fig. 1. MERIDIAN system architecture, showcasing the client, frontend, authentication, backend, database, external service, and OCR/analytical layers with the end-to-end data flow.

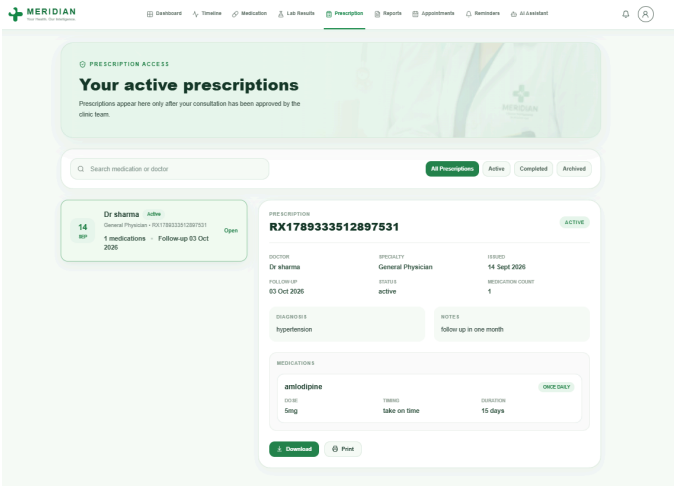
End-to-end, a typical interaction with MERIDIAN comprises eight steps, namely: (1) the user logs in to the Firebase Authentication; (2) the user is redirected to the role-specific dashboard according to the role-based access control; (3) the user performs an action, for instance, booking an appointment or uploading a report; (4) the request is processed by the Node.js/Express backend APIs; (5) the resulting data is then securely persisted in MongoDB Atlas; (6) the external services are then invoked, where applicable, such as the email notifications through the Nodemailer or the AI-generated responses, powered by the LLM service; (7) if the action involves a document, it is processed by the Python OCR service; and (8) the data is then reflected back to the user in the frontend dashboard.

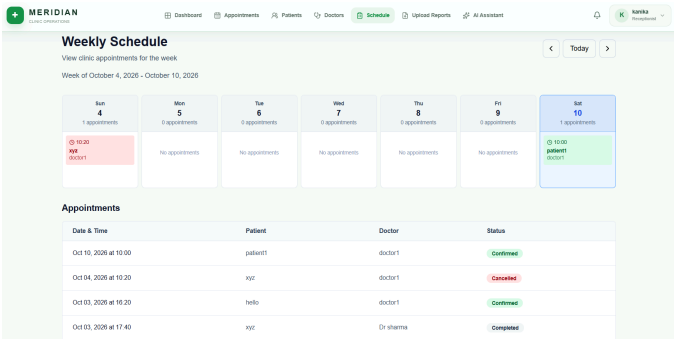
V. RESULTS AND DISCUSSION

Validation is intended to take place on patient data sets that represent chronic disease scenarios such as diabetes and hypertension, allowing the analytical modules to be exercised against non-sensitive longitudinal records. Six measures will be used to assess the system: the accuracy of laboratory trend detection across visits; the correctness of drug–drug and drug–disease interaction alerts relative to a reference interaction dataset; the accuracy of duplicate test detection within a clinically relevant time window; the response time of the decision engine when generating risk assessments; the usability of the longitudinal timeline interface as evaluated in structured walkthroughs; and the effectiveness of the red-flag risk summaries in surfacing clinically significant issues without excessive false positives.

Preliminary analysis at the design level indicates that separation of the engine from the Node.js application layer by means of a Python microservice offers two advantages that are anticipated to be reflected in the evaluation: it isolates OCR and rule-based reasoning workloads from request-handling logic, and should allow API response times to remain predictable as the interaction dataset and reference-range tables grow; and it allows the analytical modules—lab trend analysis, interaction checking, allergy screening, contraindication checking, duplicate detection and overdue-test reminders—to be extended or retrained independently of the web application. The organization of records into a timeline is also expected to facilitate demonstrating scenarios on synthetic cases that a single-visit EHR view would miss. For example a worsening renal function trend may only become visible across several laboratory results.

The main aspects of interest for evaluation of the system's performance are the way the analysis following OCR is performed for scanned and photo reports, the completeness of the drug interaction and reference range data that the system uses and the effectiveness of the fake data we generate for patient records that are not clean. Table II lists six evaluation methods. Explains how each will be checked. These aspects set the limits for the testing plan for the Capstone Project–II where the system will be tested using the six methods and improved according to the results indicating how accurate and usable it is.





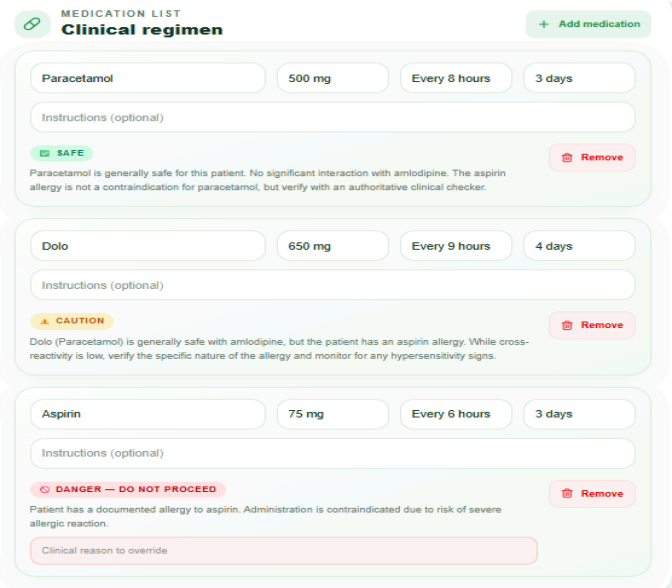


TABLE II
PROPOSED EVALUATION MEASURES FOR MERIDIAN

Metric	Evaluation Method	Target
Lab trend detection accuracy	Compare flagged trends against labeled synthetic time-series	High precision/recall
Interaction alert correctness	Cross-check alerts against reference interaction dataset	Low false-negative rate
Duplicate test detection	Inject known duplicate cases into synthetic records	High detection rate
Decision engine response time	Measure API latency under simulated load	Near real-time
Timeline usability	Structured walkthroughs with sample users	Positive feedback
Red-flag summary effectiveness	Review flagged vs. unflagged synthetic cases	Few false positives

Another thought about using of MERIDIAN is the way how the tool evolves over time the clinical practice of physicians compared to the EHR screens. The timeline, checks for interactions and summary of flags are all automatically updated once new prescription or test is added to the patient's record. It means that physician does not need to browse through a patient's encounters to cross-reference an order prior to submitting a new treatment. It should be beneficial and time-saving for physicians who have a great number of encounters for chronic disease patients. Since at the moment MERIDIAN relies on logic and not on data analysis to suggest or remind of treatments it is easy to follow the decision-making process and trace each alert to the specific rule and data triggering it. It is essential capability in context of the system designed to support rather than replace the human expertise.

VI. LIMITATIONS AND FUTURE DIRECTIONS

Although MERIDIAN brings clinical-support functions together in one platform, the current version has some limitations that must be taken into account when interpreting its results. Important limitations include: information quality available to the system, predefined rules and reference information for the modules, and considerations related to system integration.

The most significant limitation is the quality of data available to the system. Medical information extracted from scanned in prescription, laboratory reports, and entered documents may contain missing data, unclear text, or different formats. An incorrect value extracted from a report could make the analysis incorrect, so the system views OCR as a data entry aid and not as a substitution for clinical verification. The extracted information can be reviewed before it takes part in the patient's record.

A second limitation is the dependence of the modules on predefined rules and reference information. Drug interaction checking, allergy screening, laboratory thresholds, and overdue-test intervals are only as effective as the rules and datasets that are behind them. Medical guidelines and monitoring intervals can change over time, while different patients may need clinical decisions. Therefore MERIDIAN is designed as a decision-support system and not as a diagnostic or prescribing system. The final call is with the healthcare professional, and they can take into account information that may not be available to the software.

The evaluation described in this paper also has a limitation in that the initial testing is based on synthetic longitudinal patient records. Synthetic data provides the opportunity to create controlled cases for testing, abnormal laboratory trends, drug interactions, and overdue investigations, without having to share sensitive patient information. However, this type of datasets cannot accurately reflect the variety found in clinical records. Actual records may have histories, different laboratory units, differences in spelling, missing dates, and information coming from several healthcare providers. For this reason, the results obtained during the evaluation should be considered a review of the system's technical functionality and not evidence of clinical effectiveness.

There are also considerations related to system integration. A longitudinal system becomes more valuable when information from healthcare providers can be brought together in a consistent format In terms of deployment, this would require appropriate interoperability mechanisms, secure data exchange and agreement on common clinical data formats. The issue of access control would become increasingly important as more organizations and clinicians are linked to the patient history. MERIDIAN therefore needs to retain role-based access and auditing to ensure that users get only the data they need for their responsibilities.

Future development will focus on improving the reliability of document processing; more advanced preprocessing of images, field level OCR validation and structured extraction of laboratory values may further reduce errors in reports. The system can also show how confident it is in the values it extracts Ask the user to verify uncertain fields before saving them. This would enable to highlight possible false fields before saving. Especially for reports with tables with handwritten notes and different formats for different laboratory tests.

The other idea is to improve the system with more clinical rules. The current design seems to be oriented primarily on the interaction check, allergy-screening contraindications, laboratory trends analysis, duplicate test detection and overdue tests detection. However, in the future, it might be necessary to use regularly updated sets of data as reference. The rule engine should be designed in such a way that different intervals and thresholds can be set for different monitoring without changing the code of the application.

Machine-learning methods offer another direction. Having a sufficient amount of reliable data for all the cases, the system would be able to identify patients at risk of negative developments or deterioration. Such models would be trained to recognize patterns and added as an additional module to the rule-based system, rather than replacing it. The rule-based checks would make the model more interpretable as there would be specific indicators that trigger alerts, thus, the reasoning behind it would be transparent to the users.

Lastly, the future research should go beyond the prototype testing. Instead, large-scale evaluations and structured usability studies should be done. System performance can be tested with patient records, and more requests, at the same time. At the time, clinical reviewers can check whether the alerts make sense and are helpful. They can also see whether the system triggers many alerts that lead to alert fatigue. These steps would help figure out if the MERIDIAN system design and its analytical methods can support long-term world decision support. It would also help to confirm that the system is transparent, reliable, and properly supervised by healthcare professionals.

VII. CONCLUSION

This paper introduces MERIDIAN – a clinical decision engine that seeks to organize fragmented patient medical data into a time-ordered timeline by adding a rule-based engine to this timeline to help physicians prescribe medication more safely. The objective is to reduce diagnostic testing and the likelihood of harmful drug reactions and improve care continuity across multiple healthcare providers, especially in places where paperwork is still the norm such as much of India.

From a computer engineering point of view MERIDIAN shows how structured document databases, chronological data modeling and rule-based decision-making can be used for healthcare-related problems. Its architecture is built using the stack and microservices which makes the analytical engine scalable and easy to expand. Future work, to be done as part of Capstone Project–II will focus on measuring the system's performance using the metrics described in Section V. This will include testing with data and when possible anonymized real-world data. The team also plans to upgrade the rule-based engine by adding machine-learning models which can predict risks early. Additionally they plan to explore connecting MERIDIAN with health information exchanges.

On a level MERIDIAN shows how interoperable longitudinal digital health systems can make a difference in multi-doctor environments. In places where records are incomplete or fragmented patients often get repeated tests, unsafe prescriptions and delayed diagnoses. For patients in such settings carrying paper prescriptions and printed lab results from one doctor to another is a common problem. In these cases an OCR-based ingestion pipeline is not a nice-to-have but essential for creating any kind of longitudinal medical record at all. By combining transformation and decision support into a unified workflow MERIDIAN aims to bring the value of clinical decision support to the environments where they are most needed – namely those which are still heavily reliant on paper.

REFERENCES

[1] Agency for Healthcare Research and Quality, "Clinical Decision Support Systems," AHRQ PSNet, U.S. Department of Health and Human Services, Rockville, MD, USA, 2019.

[2] R. T. Sutton, D. Pincock, D. C. Baumgart, D. C. Sadowski, R. N. Fedorak, and K. I. Kroeker, "An overview of clinical decision support systems: Benefits, risks, and strategies for success," npj Digital Medicine, vol. 3, art. no. 17, 2020, doi: 10.1038/s41746-020-0221-y.

[3] T. J. Loftus, J. A. Balch, J. L. Marquard, J. M. Ray, B. S. Alper, N. Ojha, A. Bihorac, G. Melton-Meaux, G. Khanna, and C. J. Tignanelli, "Longitudinal clinical decision support for assessing decisions over time: State-of-the-art and future directions," Digital Health, vol. 10, 2024, doi: 10.1177/20552076241249925.

[4] C. Scherkl, T. Dierkes, M. Metzner, D. Czock, H. M. Seidling, W. E. Haefeli, and A. D. Meid, "Towards a prescribing monitoring system for medication safety evaluation within electronic health records: A scoping review," BMC Medical Informatics and Decision Making, vol. 25, art. no. 244, 2025, doi: 10.1186/s12911-025-03096-3.

[5] A. Syrowatka, A. Motala, E. Lawson, and P. G. Shekelle, "Computerized Clinical Decision Support to Prevent Medication Errors and Adverse Drug Events," Making Healthcare Safer IV: A Continuous Updating of Patient Safety Harms and Practices, AHRQ Publication No. 24-EHC019–6, Rockville, MD, USA, 2024, doi: 10.23970/AHRQEPC_MHS4MEDERROR.

[6] M. Felisberto, G. dos Santos Lima, I. C. Celuppi, M. dos Santos Fantonelli, W. L. Zanotto, J. M. Dias de Oliveira, E. T. B. Mohr, R. A. dos Santos, D. H. Scandolara, C. L. Cunha, J. F. Hammes, J. S. da Rosa, I. G. Demarchi, R. S. Wazlawick, and E. M. Dalmarco, "Override rate of drug-drug interaction alerts in clinical decision support systems: A brief systematic review and meta-analysis," Health Informatics Journal, vol. 30, no. 2, 2024, doi: 10.1177/14604582241263242.

[7] R. Smith, "An overview of the Tesseract OCR engine," in Proc. 9th International Conference on Document Analysis and Recognition (ICDAR), IEEE Computer Society, 2007, pp. 629–633, doi: 10.1109/ICDAR.2007.4376991.

[8] A. M. Holbrook, J. Matos Silva, J. A. Y. Faruque, J. Deng, T. Schneider, and A. Jaffer, "Effect of electronic drug-drug interaction alerts on patient and clinician outcomes: A systematic review," Journal of the American Medical Informatics Association, vol. 32, no. 10, pp. 1617–1628, 2025, doi: 10.1093/jamia/ocaf139.